

자기소개서

Applied AI Engineer - Agent GYM — Solar Agent Engineering 팀

이기찬 | 010-6666-2824 | gichanlee@icloud.com |

포트폴리오: <https://bit-habit.com/?title=Applied%20AI%20Engineer> | GitHub: github.com/bookseal

1. "실제로 작동하게 만드는" 일을, 이미 혼자 완주해봤습니다

채용 공고는 이 포지션을 "연구 성과를 안정적이고 확장 가능한 시스템으로 전환하며, AI 기술을 실제로 작동하게 만드는 엔지니어링 실행의 중심" 이라고 정의합니다.

저는 그 전환을 **프로토타입에서 배포·운영까지 혼자 완주**해봤습니다.

BookToss(booktoss.bit-habit.com)는 서울 25 개 구립도서관을 통합 검색하는 LangGraph 에이전트입니다. 문제는 **도서관들이 공개 API 를 주지 않는다**는 것이었습니다. Browser-use/Playwright 로 각 도서관 웹을 도구(tool)로 감싸고, Tool Calling 으로 에이전트가 25 곳을 조회하게 만들었습니다. 실패와 재시도는 상태 그래프로 다뤘습니다. 도서 탐색 시간이 **5 분에서 1 분으로** 줄었습니다.

이 에이전트는 지금도 제가 운영하는 k3s 클러스터 위에서 Prometheus 로 관측되고 ArgoCD 로 배포되며 돌아가고 있습니다. **데모에서 멈춘 에이전트와 살아있는 에이전트 사이의 거리**를 저는 직접 걸어봤습니다.

2. 감이 아니라 측정으로 고릅니다

채용 공고의 우대 사항에 "품질 평가(Evals), 회귀 테스트, 실험 설계" 가 있었습니다. 저는 이 문장을 보고 **제가 RAG 콘테스트에서 실제로 한 일**을 떠올렸습니다.

미국 항공법(14 CFR) **1,297 쪽**을 근거로 §를 인용해 답하는 RAG 를 만들었습니다. 대부분의 참가자가 연습 문제 5 개를 눈으로 보며 튜닝할 때, 저는 **밤새 45 개 설정을 자동으로 돌리는 eval 하네스**를 짰습니다 — 청킹 × 임베딩 × 검색 방식 × K 를, blind 14 문항 홀드아웃에 대고 coverage·recall·MRR 로 채점하고, **인용한 §가 실제로 그 청크에 존재하는지까지 프로그램으로 검증**했습니다.

배포본은 느낌이 아니라 규칙으로 골랐습니다 — "노이즈 범위 안이면 더 싼 쪽을 택한다." 그 규칙이 top-coverage 설정보다 **토큰을 38% 아끼는** 구성을 선택하게 했습니다. 같은 원칙으로 **더 잘 검색하는 agentic 루프를 죽였습니다**(15,954 토큰 vs single-shot 2,256). 좋은 걸 만드는 것보다, 비용을 못 갚는 좋은 것을 안 쓸 줄 아는 게 더 어렵습니다. 이 과제는 **강사(Larry Arnstein) 채점**에서 **9/10, 수강생 중 1 등**을 받았습니다.

AI 시스템을 "돌아가게" 만드는 것과 "믿을 수 있게" 만드는 것은 다릅니다. 후자는 반드시 **측정**을 통과해야 합니다.

3. 관측되지 않는 에이전트는 작동하는 것이 아닙니다

채용 공고 필수 사항의 "LLM/Agent 시스템의 observability(trace, log, metric)" 는, 에이전트를 실제로 운영해 본 사람만 그 무게를 안다고 생각합니다.

에이전트는 실패하는 방식이 특이합니다. **죽지 않고 그럴듯하게 틀립니다.** 도구 호출이 빈 결과를 반환했는데 모델이 그럴듯한 답을 지어내면, 로그에는 200 OK 만 남습니다. 실패가 **조용히 성공으로 위장**됩니다. 그래서 에이전트에서 관측은 선택이 아니라 **작동의 정의 자체**라고 생각합니다.

Solar 로 BookToss 를 재구축하면서 이 문제를 실제로 만났습니다. 모델이 답이 **"근거 있는 사실"인지 "그럴듯한 추측"인지 스스로 구분해 표시하지 못했고**, 불확실할 때 보류(abstain)하는 장치가 약했습니다. 저는 이것을 불평이 아니라 **재현 절차가 붙은 공개 피드백**으로 정리했습니다 (bookseal.github.io/Booktoss/feedback.html). 제가 운영하는 14 개 서비스도 Prometheus 로 관측하며, 상태를 외부에 공개해 두었습니다(status.bit-habit.com) — **깨져 있으면 깨진 채로 보이도록.**

4. 연구를 제품으로 옮기는 일 = 실행 표준을 만드는 일

채용 공고는 이 팀이 "전사적으로 AI 를 규모 있게 실행하기 위한 엔지니어링 표준을 세운다" 고 말합니다. 저는 이 문장을 제 현직 업무의 정의로 읽었습니다.

한국경영분석연구원에서 **Claude Code·Codex 에이전트를 사내 개발 프로세스 자체에 이식**했습니다. GitHub Issue 생성부터 PR·코드 리뷰까지 자동화했고, 정부 소프트웨어 대가 산정 지침서 검토를 LLM 워크플로우로 재설계해 **10 시간 → 2 시간**으로 줄였습니다.

그런데 가장 신경 쓴 것은 단축된 시간이 아니라, **비전공자 팀원이 저 없이도 돌릴 수 있게 문서로 남기는 일**이었습니다. 자동화가 저에게만 의존하면 그건 **새로운 병목**이기 때문입니다. 사내 도구(quali-fit)를 만들 때도 도메인 로직을 UI·DB 에서 분리하고 단위 테스트로 고정했습니다 — 나중에 프론트를 FastAPI 로 갈아끼워도 재작성이 없도록. "**기술 전파 및 표준화**" 는 제가 아무도 시키지 않았을 때부터 해오던 일입니다(42 스테디 3 년, wiki.bit-habit.com 가이드북 공개, LLM App Lab 강의·코드 전체 공개).

5. 부족한 부분은 감추지 않겠습니다

- **연구 경험(논문·모델 학습)이 없습니다.** 저는 모델을 만드는 사람이 아니라, 만들어진 모델을 **실제로 돌아가게 만드는 사람**으로 지원합니다. 채용 공고가 정의한 Applied AI Engineer 가 정확히 그 역할이라고 이해했습니다.
- **GPU 서빙·분산 학습 실무 경험이 없습니다.** 온디맨드 서빙 구조(k3s scale-to-zero)는 직접 만들었지만 CPU 환경이었습니다.
- **개발 연차가 길지 않습니다.** 다만 그 시간에 에이전트를 만들어 배포하고 2 년을 운영했고, RAG 를 45 개 설정으로 측정해 1 등을 했고, 글로벌 기술지원에서 20 명 중 전 지표 1 위를 했습니다.

제가 만든 것들은 지금 이 순간에도 전부 켜져 있습니다. **연차로 증명할 수 없다면, 굴러가는 것으로 증명하겠습니다.**

포트폴리오 — <https://bit-habit.com/?title=Applied%20AI%20Engineer>

에이전트 프로젝트, LLM 앱 런처, RAG eval 하네스와 각 저장소로 연결됩니다.